Drought Relief Targeting: Team Study Guide

Team Geek Squad · DSU "Data to Harvest" Hackathon · Sunday, October 18, 2026
Live site: `https://ms-cc-org.github.io/DSU-Hackathon-2026-Geek-Squad/` · Code as of commit `39d86e6`

Every team member must be able to explain and change any part of the code during judging. This guide covers how the system fits together, what each file does, how to change things live, and what each change will do to the results.

1. How the pieces connect
2. The model, step by step
3. The API server
4. The website
5. The AI assistant
6. Changing code live during the demo
7. Practice drills and expected results
8. Questions judges are likely to ask

---

1. How the pieces connect

```
data/master_dataset.parquet   one row = one county × crop × year (17,056 rows, 4 states, 2000–2025)
        │  load_master()  ← src/data_loader.py
        ▼
src/priority_model.py         THE MODEL: score → dollar loss → who gets money → blind test
src/insights.py               shared settings (test years, "disaster" threshold) + correlations
src/report_card.py            one county's 26-season history
        │                                  │
        ▼                                  ▼
api_server.py  (JSON at /api/*)        app.py  (Streamlit backup, calls the same functions)
        │   └── src/advisor.py  (AI chat: Claude calls 5 of our functions as "tools")
        ▼
frontend/  (React)   every number on screen comes from the API; nothing is calculated in JavaScript
```

- **Hosting:** the website is on GitHub Pages; the Python API is on Render. A push to `main` redeploys both automatically in about 2–5 minutes. Opening the Render address redirects to the website.
- **The dataset** covers Iowa, Nebraska, California and Delaware. Iowa and Nebraska corn and soybeans have the fullest records; California, Delaware and sorghum are thin, and the app says so.

> **One sentence for judges:** "The math lives in one Python file. Both dashboards and the AI call it, so they can never disagree."

2. The model, step by step (src/priority_model.py)

urgency_components(), line 31: turns August data into a hazard number

1. **Scale each input from 0 to 1** against a fixed cap. Missing values count as 0 (no stress).
   - Heat days ≥ 35 °C ÷ 15 (line 40)
   - Weeks in severe drought (D2+) ÷ 20 (line 39)
   - Drought severity index (DSCI) ÷ 500 (line 38)
   - Rainfall shortfall ÷ 50%, counting only *below-average* rain (line 43)

2. **Hazard** = 0.30 × heat + 0.30 × drought weeks + 0.25 × DSCI + 0.15 × rain shortfall (line 57). The weights add up to 1.
3. **Soil discount:** soil water storage ÷ the 95th percentile of the counties being scored. Net = hazard × (1 − 0.25 × soil) (line 65), so deep soil cuts the score by up to 25%.

compute_urgency_score(), line 79

Score = 1 + 9 × net, rounded to one decimal (line 95). 1 means no stress; 10 is the maximum.

calculate_economic_losses(), line 99: bushels into dollars

1. Trend yield = actual yield ÷ (1 + yield anomaly).
2. Lost bushels = (trend − actual) × harvested acres.
3. Dollar loss = lost bushels × USDA price (corn $4.75/bu, soybeans $12.20, wheat $6.50, sorghum $4.50; line 23).

allocate_relief_budget(), line 147: who gets money

1. Takes one state, crop and season (reported yields only) and ranks counties by score.
2. Ties are broken by the unrounded score, then FIPS code (`_top_first`, line 138). Never by dollar loss, which isn't known until harvest.
3. Tier 1 = top 15% (rounded up). Tier 2 = next 20% (line 197). Everyone else is Tier 3.
4. The fund goes only to Tier 1, in proportion to each county's dollar loss. If no Tier 1 county had a loss, it's split by score × acres.

evaluate_out_of_sample_validation(), line 236: the blind test

1. For each season 2016–2025, rank counties four ways: our score, rainfall alone, DSCI alone, random picks.
2. Fund the top 15% under each method.
3. A **hit** is a funded county whose harvest came in at least 10% below trend.
4. Precision = hits ÷ funding slots, over seasons with at least one disaster.

Shared settings: src/insights.py

- Test years: lines 12–13 (rules fixed; tested on 2016–2025).
- What counts as a disaster, −10%: line 14.
- Minimum test size before the app claims a win (30 slots): line 15. Below that it says "can't test fairly" (e.g. Delaware).

Headline results (Iowa corn, $25M fund, top 15%)

Method	Precision 2016–2025	Hits
Our urgency score	**20.7%**	23
Drought Monitor (DSCI) alone	18.0%	20
Rainfall alone	15.3%	17
Random picks	8.8%	≈ 9.7

That is 1.35× better than rainfall. Nebraska corn: 22.6% (24 hits) vs 17.0% for rainfall, 1.33×. 2020 example: 15 of 95 Iowa counties funded, trigger score 3.7, Guthrie #1 (score 4.8), Carroll gets the largest check ($3.25M), Tama lost $44.7M but scores 1.5 (rank #59, derecho windstorm, not drought) and gets $0.

3. The API server (api_server.py)

- **No web framework**, just Python's built-in HTTP server.
- **Loaded once at startup:** the dataset (`DF`, line 57) and the AI assistant (`ADVISOR`).
- **Routing:** `route()` (line 317) matches each URL to a function: options, ranking, ranking CSV, validation, report card, county map, quota sweep. `do_POST` handles `/api/chat`.
- **Slider limits** in `options()`: fund $5M–$100M, quota 5%–30% (lines 133–134).
- **Hosting settings:** `CORS_ORIGINS` lets only our GitHub Pages site call the API from a browser; `FRONTEND_URL` makes the Render address redirect to the site.
- **The API key** is read from `.env` once at startup (never committed). Restart the server after changing it.

4. The website (frontend/src)

- **App.jsx** holds all the state: whether the landing screen is showing, the current page, state, crop, season, fund ($25M), quota (15%), the selected county and the chat history. It fetches four endpoints with `useApi()`, which refetch automatically when a setting changes.
- **Header.jsx:** the `PAGES` list (tab order) and the controls. Clicking the title returns to the landing screen.
- **lib/api.js:** builds the API URLs, using the Render address on the live site.

Page	Component
Opening screen (globe, two buttons)	`Landing.jsx`
1 · The decision	`DecisionSection.jsx`
2 · Who gets relief (map, ranking)	`AnswerSection.jsx` + `CountyMap.jsx`
3 · Why	`WhySection.jsx`
4 · Does it work?	`ProofSection.jsx`
5 · County report card ("Show the math")	`ReportCardSection.jsx` + `RationaleModal.jsx`
6 · Limits & data	`LimitsSection.jsx`
Ask the data (AI)	`AdvisorView.jsx`

5. The AI assistant (src/advisor.py)

- **It never sees the raw data.** Claude (`claude-opus-5-5`, line 24) can only call five read-only Python functions: `rank_counties`, `county_details`, `validation_results`, `what_drives_losses` and `data_coverage`. They run the same model code as the dashboard, so every number it quotes matches the screen.
- **The system prompt** (line 27) explains the model and tells Claude to get every number from a tool, to say "The data can't answer that" for field-level, irrigation, other-state or post-2025 questions, and to warn when a sample is small.
- **Loop:** the SDK's `tool_runner` calls the tools until Claude has its answer. Effort is set to `low` for quick chat replies (line 239).

6. Changing code live during the demo

> **Don't edit the live site during judging.** A push takes 2–5 minutes to appear. Make changes on a laptop, where they show up in seconds.

Setup before judging (two terminals, both with `.venv` active)

```
python api_server.py                    # terminal 1: the API on :8000
cd frontend && npm run dev              # terminal 2: open http://localhost:5173
```

Frontend edits appear the moment you save. After any **Python** edit, press Ctrl+C in terminal 1 and run `python api_server.py` again (about 10 seconds).

If a judge says…	Change this	Then
"Make heat count more"	`priority_model.py` line 57: `0.30 * heat_norm`	restart API
"Ignore soil"	line 65: `0.25` → `0`	restart API
"Change the heat cap"	line 40: `/ 15.0`	restart API
"Define disaster as 20% below trend"	`insights.py` line 14: `-10.0` → `-20.0`	restart API
"Corn is $4.25 now"	`priority_model.py` line 23: price	restart API
"Start on a $50M fund"	`App.jsx` line 24: `useState(25)`	instant
"Allow funding up to 50% of counties"	`api_server.py` line 134: `max: 30`	restart API
"Make Tier 2 wider"	`priority_model.py` line 197: `+ 0.20`	restart API
"Change the wording on page 1"	`DecisionSection.jsx`	instant
"Add a suggested AI question"	`AdvisorView.jsx` line 25 list	instant
"Make the AI answer shorter"	`advisor.py` line 27, system prompt rule 5	restart API

> **Undo everything afterwards:** `git checkout -- .` restores the committed version. Nothing is at risk unless you commit.

7. Practice drills and expected results

Each change below was run on a temporary copy of the model (Iowa corn, blind test 2016–2025), so you know what the "Does it work?" page should show.

Change	Our model	Rainfall	DSCI	2020 trigger	What to say
Current code	**20.7%**	15.3%	18.0%	3.7	—
Heat weight → 0	18.0%	15.3%	18.0%	3.4	"Heat is what puts us ahead of the drought monitor."

Rain weight → 0.45	18.0%	15.3%	18.0%	5.2	"More rainfall weight makes it worse, as the correlations predicted."
Disaster = –20%	**23.3%**	13.3%	16.7%	3.7	"The lead grows for the worst disasters."
Fund top 30%	13.8%	11.9%	**14.7%**	3.2	See the warning below.

> **Know the last row before the demo.** If a judge drags the quota slider to 30%, the drought monitor alone edges ahead (14.7% vs 13.8%). Our model wins for **targeted** relief, the top 15%. Once you fund almost a third of the state, every method is mostly picking counties that weren't hit, and the edge disappears. Say this plainly.

Guthrie stays #1 in every scenario, a good sign the ranking holds up.

Drill: with `npm run dev` running, make each of the first four changes, check the "Does it work?" page against this table, then run `git checkout -- .`. Repeat until each change takes under a minute.

8. Questions judges are likely to ask

"How did you choose the weights?"

We ranked the inputs by how strongly each tracks yield loss: heat days ($r = -0.52$) and weeks in severe drought (–0.37) get 0.30 each, drought severity (–0.36) gets 0.25, rainfall (+0.13) only 0.15. Soil has almost no link on its own (≈ 0), so it discounts the score instead of adding to it. The weights are set by hand, not fitted, and the correlations use all 26 years. What the blind test proves is that these frozen weights beat the baselines on 2016–2025.

"Your model loses some seasons."

True, and the page shows every season. In Nebraska 2022 rainfall alone caught 9 disasters and we caught 3. Over all ten test years we win in both states: 23 vs 17 hits in Iowa, 24 vs 18 in Nebraska. In 2018, six Iowa counties had severe losses and every method caught zero: those losses didn't follow drought signals.

"Why does Tama get nothing after losing $44.7M?"

The fund is for drought, and Tama's loss wasn't drought-driven. It scores 1.5: few heat days, little severe drought. The August 2020 derecho windstorm flattened crops there. Our score separates the two.

"Can I check any number?"

Yes. On the county report card, "Show the math" shows every input, weight and step for the score, the dollar loss and the relief check. The same values are in the CSV export.

"Why not a neural network?"

A state official distributing public money must defend every dollar to auditors and farmers. Our score is transparent and tied to measured facts: heat days, drought duration and soil water.

"Did you use AI?"

Yes, disclosed in section 11 of `readmeforGroup.md`: Gemini for brainstorming, Antigravity for the first part of the build, Claude for the team notes, and Claude Code for the later work. The in-app assistant only quotes numbers it gets from our own model functions.